

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de gui3n.

Contenidos multimedia

V3deo

Video en la web

El vídeo es un componente fundamental en la experiencia digital actual. Desde contenido educativo hasta publicidad interactiva, la correcta integración de vídeo en la web permite mejorar la experiencia del usuario y optimizar la carga y visualización de los recursos multimedia.

Formatos de vídeo y comparativa de calidad

Los principales formatos de vídeo utilizados en la web son:

Formato	Códec	Ventajas	Desventajas
MP4	H.264 / H.265	Alta compatibilidad, buena calidad-tamaño	Puede no ser el mejor en compresión
WebM	VP8 / VP9	Optimizado para la web, mejor compresión	No compatible con todos los dispositivos
AVI	Diversos	Alta calidad	Archivos muy grandes
MKV	Diversos	Soporta múltiples pistas de audio y subtítulos	No estándar en navegadores
AV1	AV1	Mejor compresión y calidad	No todos los navegadores lo soportan aún

- **Ejercicio:** Buscar un mismo vídeo en diferentes formatos y analizar su calidad y tamaño en MB.

Repositorios de vídeo

Para obtener vídeos libres de derechos, se pueden utilizar los siguientes recursos:

- [Pexels](#)
 - [Videvo](#)
 - [Pixabay](#)
 - [Mixkit](#)
 - [Artgrid](#) (premium)
-
- **Ejercicio:** Descargar un vídeo libre de derechos y analizar sus metadatos (formato, tamaño, resolución).

Reproductores de vídeo e inserción en navegadores web

El elemento <video> de HTML5 permite incrustar vídeos de forma nativa:

Podemos personalizar el reproductor con los atributos nativos de la etiqueta:

Atributo	Descripción	Valores posibles
src	Especifica la URL del vídeo (se usa dentro de <video>, pero es más común dentro de <source>)	URL del archivo de vídeo
controls	Muestra los controles predeterminados del reproductor (play, pausa, volumen, etc.)	No tiene valor, se usa solo con su nombre
autoplay	Hace que el vídeo se reproduzca automáticamente al cargar la página	No tiene valor, se usa solo con su nombre
loop	Hace que el vídeo se reinicie automáticamente al finalizar	No tiene valor, se usa solo con su nombre
muted	Inicia el vídeo con el audio silenciado	No tiene valor, se usa solo con su nombre
poster	Imagen de previsualización antes de que el vídeo cargue o se reproduzca	URL de una imagen (.jpg, .png, etc.)
preload	Indica cómo debe cargarse el vídeo antes de la reproducción	auto, metadata, none
width / height	Define el tamaño del vídeo en píxeles (aunque es preferible gestionarlo con CSS)	Número en píxeles

Ejemplo html con atributos:

```
<video controls width="600" poster="preview.jpg" preload="metadata">  
  <source src="video.mp4" type="video/mp4">  
  <source src="video.webm" type="video/webm">  
  Tu navegador no soporta el elemento de vídeo.  
</video>
```

Notas importantes:

- Si usas autoplay, muchos navegadores requieren que también esté muted, de lo contrario, el vídeo no se reproducirá automáticamente.
- loop es útil para fondos de vídeo o animaciones repetitivas.
- preload="metadata" es una opción intermedia entre carga rápida y disponibilidad de información como duración del vídeo.

Para personalizar la reproducción, se pueden utilizar librerías de terceros como:

- **Plyr.js**: Ligero y accesible.
- **Video.js**: Ampliamente utilizado, con muchas opciones.
- **MediaElement.js**: Compatible con múltiples formatos.

Controlando la etiqueta <video> con JavaScript

HTML5 introdujo la etiqueta <video> como una forma nativa de incrustar videos en la web sin depender de Flash. Aunque la etiqueta proporciona controles básicos (controls), con JavaScript podemos crear una experiencia personalizada, añadiendo botones, controles avanzados y efectos interactivos.

Propiedades y Métodos de <video> en JavaScript

Para manipular un video con JavaScript, primero debemos obtener la referencia al elemento <video> usando document.querySelector() o getElementById().

Ejemplo js:

```
const video = document.querySelector("#miVideo");
```

- Una vez tenemos el elemento, podemos usar sus propiedades y métodos:

Propiedad/Método	Descripción
video.play()	Inicia la reproducción.
video.pause()	Pausa el video.
video.currentTime	Obtiene o establece el tiempo de reproducción (en segundos).
video.duration	Devuelve la duración total del video.
video.volume	Controla el volumen (valor entre 0 y 1).
video.muted	Silencia el video (true o false).
video.playbackRate	Cambia la velocidad de reproducción (ejemplo: 1.0 es normal, 0.5 es más lento).
video.requestFullscreen()	Muestra el video en pantalla completa.

- **Ejercicio:** Crear una página con un vídeo incrustado usando `<video>` y personalizar los controles con JavaScript.

Agregando controles personalizados

Ahora crearemos un conjunto de botones personalizados para controlar un video.

html:

```
<video id="miVideo" width="600" poster="poster.jpg">
  <source src="video.mp4" type="video/mp4">
  Tu navegador no soporta el elemento de video.
</video>

<div class="controles">
  <button id="playPause"><img alt="play/pause icon" data-bbox="333 408 353 433"/> Play</button>
  <button id="stop"><img alt="stop icon" data-bbox="333 438 353 458"/> Stop</button>

  <button id="mute"><img alt="mute icon" data-bbox="288 468 308 493"/> Mute</button>
  <button id="fullscreen"><img alt="fullscreen icon" data-bbox="333 503 353 523"/> Pantalla completa</button>
  <input type="range" id="volumen" min="0" max="1" step="0.1" value="1">
</div>
```

CSS:

```
.controles {
  margin-top: 10px;
  display: flex;
  gap: 10px;
}

button {
  padding: 8px 12px;
  cursor: pointer;
}
```

js:

```
const video = document.getElementById("miVideo");
const playPauseBtn = document.getElementById("playPause");
const stopBtn = document.getElementById("stop");
const muteBtn = document.getElementById("mute");
const fullscreenBtn = document.getElementById("fullscreen");
const volumenSlider = document.getElementById("volumen");

// ▶ Play/Pause
playPauseBtn.addEventListener("click", () => {
  if (video.paused) {
    video.play();
    playPauseBtn.textContent = "⏸ Pause";
  } else {
    video.pause();
    playPauseBtn.textContent = "▶ Play";
  }
});

// ■ Stop (Reinicia el video)
stopBtn.addEventListener("click", () => {
  video.pause();
  video.currentTime = 0;
  playPauseBtn.textContent = "▶ Play";
});
```

```
// 🎧 Mute/Unmute
muteBtn.addEventListener("click", () => {
    video.muted = !video.muted;
    muteBtn.textContent = video.muted ? "🔊 Unmute" : "🔇 Mute";
});
```

```
// 🖥️ Pantalla Completa
fullscreenBtn.addEventListener("click", () => {
    if (video.requestFullscreen) {
        video.requestFullscreen();
    }
});
```

```
// 📊 Control de Volumen
volumenSlider.addEventListener("input", () => {
    video.volume = volumenSlider.value;
});
```

Interacciones Avanzadas

- **Control de Progreso del Video**

Podemos mostrar una barra de progreso del video usando `timeupdate` y `currentTime`.

- html

```
<progress id="barraProgreso" value="0" max="100"></progress>
```

- js

```
const barraProgreso = document.getElementById("barraProgreso");

video.addEventListener("timeupdate", () => {
  const porcentaje = (video.currentTime / video.duration) * 100;
  barraProgreso.value = porcentaje;
});
```

- **Velocidad de Reproducción**

Podemos permitir que el usuario cambie la velocidad del video con un select.

- html

```
<label for="speed">Velocidad:</label>
<select id="speed">
  <option value="0.5">0.5x</option>
  <option value="1" selected>1x</option>
  <option value="1.5">1.5x</option>
  <option value="2">2x</option>
</select>
```

- js

```
const speedSelector = document.getElementById("speed");

speedSelector.addEventListener("change", () => {
  video.playbackRate = speedSelector.value;
});
```

- **Adelantar y Retroceder Video**

Los videos en HTML5 tienen la propiedad `currentTime`, que nos permite modificar la posición de reproducción.

- html

```
<video id="miVideo" controls width="600">
  <source src="video.mp4" type="video/mp4">
</video>

<button id="retroceder">⏮ Retroceder 10s</button>
<button id="avanzar">⏭ Avanzar 10s</button>
```

- js

```
const video = document.getElementById("miVideo");
const btnRetroceder = document.getElementById("retroceder");
const btnAvanzar = document.getElementById("avanzar");

btnRetroceder.addEventListener("click", () => {
  video.currentTime -= 10;
});

btnAvanzar.addEventListener("click", () => {
  video.currentTime += 10;
});
```

 **¿Cuándo usarlo?** Ideal para mejorar la accesibilidad y navegación del usuario.

- **Modo Picture-in-Picture (PiP)**

El modo Picture-in-Picture permite que el video se reproduzca en una mini ventana flotante mientras el usuario navega por otra página.

- html

```
<button id="pip"> Activar PiP</button>
```

- js

```
const btnPiP = document.getElementById("pip");

btnPiP.addEventListener("click", async () => {
  if (document.pictureInPictureElement) {
    document.exitPictureInPicture();
  } else {
    await video.requestPictureInPicture();
  }
});
```

 **Nota:** Esto funciona en Chrome, Edge y Firefox, pero Safari lo maneja diferente.

- **Control de Subtítulos en <track>**

HTML5 permite añadir subtítulos con la etiqueta <track>, pero podemos activarlos/desactivarlos con JavaScript.

- html

```
<video id="miVideo" controls width="600">  
  <source src="video.mp4" type="video/mp4">  
  <track id="subtitulos" kind="subtitles" srclang="es" src="subtitulos.vtt" label="Español" default>  
</video>
```

- js

```
const btnSubtitulos = document.getElementById("toggleSubs");  
const track = document.getElementById("subtitulos");  
  
btnSubtitulos.addEventListener("click", () => {  
  const textTrack = video.textTracks[0]; // Obtiene el primer track  
  textTrack.mode = textTrack.mode === "showing" ? "hidden" : "showing";  
});
```

¿Cómo funciona?

- textTracks[0] obtiene la primera pista de subtítulos.
- mode = "showing" activa los subtítulos, mode = "hidden" los oculta.

Streaming y plataformas de vídeo en la web

Existen dos formas principales de incrustar videos en una web:

1. Plataformas externas (YouTube, Vimeo, Twitch) con `<iframe>`.
2. Streaming propio con tecnologías como HLS y MPEG-DASH.

1. Incrustar vídeos de YouTube con `<iframe>`

Un vídeo de YouTube se puede insertar con:

```
<iframe width="560" height="315"  
  src="https://www.youtube.com/embed/VIDEO_ID"  
  frameborder="0"  
  allowfullscreen>  
</iframe>
```

Pero hay opciones avanzadas de personalización con parámetros en la URL.

Opciones de personalización:

- `autoplay=1` → Reproduce el vídeo automáticamente.
- `controls=0` → Oculta los controles del reproductor.
- `loop=1` → Repite el vídeo en bucle.
- `mute=1` → Inicia el vídeo silenciado.
- `start=30` → Comienza la reproducción en el segundo 30.
- `rel=0` → Evita mostrar vídeos recomendados al final.

Lista completa de parámetros:

https://developers.google.com/youtube/player_parameters

Ejemplo con personalización:

```
<iframe width="560" height="315"  
  src="https://www.youtube.com/embed/VIDEO_ID?autoplay=1&controls=0&loop=1&mute=1&start=30"  
  frameborder="0"  
  allowfullscreen>  
</iframe>
```

2. Streaming Propio: HLS y MPEG-DASH

Cuando necesitas transmitir video en vivo o bajo demanda, plataformas como YouTube y Twitch no siempre son suficientes. HLS y MPEG-DASH permiten el streaming adaptable.

HLS (HTTP Live Streaming)

- Creado por Apple.
- Divide el video en pequeños segmentos (.ts).
- Necesita un archivo índice (.m3u8).
- Soportado en Safari, iOS y la mayoría de navegadores con librerías como hls.js.

Ejemplo de video HLS con JavaScript (hls.js)

```
<video id="video" controls></video>
<script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></script>
<script>
  var video = document.getElementById('video');
  if (Hls.isSupported()) {
    var hls = new Hls();
    hls.loadSource('https://test-streams.mux.dev/x36hzz/x36hzz.m3u8'); // URL de ejemplo
    hls.attachMedia(video);
  }
</script>
```

MPEG-DASH

- Alternativa a HLS, pero de código abierto.
- Utilizado por YouTube y Netflix.
- Fragmenta los vídeos en segmentos .mp4 con un archivo .mpd de control.
- Compatible con DASH.js.

Ejemplo de video MPEG-DASH con dash.js

```
<video id="video" controls></video>
<script src="https://cdn.dashjs.org/latest/dash.all.min.js"></script>
<script>
  var player = dashjs.MediaPlayer().create();
  player.initialize(document.querySelector("#video"),
    "https://dash.akamaized.net/envivio/EnvivioDash3/manifest.mpd",
    true);
</script>
```

¿Cuándo usar HLS o MPEG-DASH?

HLS → Mejor compatibilidad con dispositivos Apple.

MPEG-DASH → Más eficiente en Android y Chrome.

Tarea para los alumnos

1. Insertar un vídeo de YouTube con opciones avanzadas.
2. Investigar cómo funcionan HLS y MPEG-DASH en diferentes navegadores.

Conversión de formatos y optimización

Para optimizar vídeos sin perder calidad:

- **FFmpeg**: Herramienta de línea de comandos para conversión.
- **HandBrake**: Software gratuito y fácil de usar.
- **Shutter Encoder**: Conversión y compresión de archivos.

Ejemplo de conversión con FFmpeg:

```
ffmpeg -i video.mp4 -c:v libvpx -b:v 1M -c:a libvorbis video.webm
```

- **Ejercicio**: Convertir un vídeo MP4 a WebM usando HandBrake y comparar la calidad y tamaño.

Edición y postproducción de vídeo

Para edición y creación de efectos:

- **DaVinci Resolve:** Gratuito y potente.
- Adobe Premiere Pro: Profesional.
- **After Effects:** Creación de efectos.
- **Kapwing / Canva:** Edición online fácil de usar.
- **Ejercicio:** Editar un vídeo corto agregando títulos y transiciones en una herramienta de edición gratuita.

Ejercicio final avanzado: Vídeo como fondo en una web

Crear una sección de una web con un vídeo como fondo, aplicando técnicas de HTML, CSS y JavaScript para una mejor experiencia de usuario.

Requisitos del ejercicio

- ✓ **Vídeo de fondo:** Un archivo de vídeo en formato .mp4 (y opcionalmente .webm para compatibilidad).
- ✓ **Texto superpuesto:** Un título y un botón de llamada a la acción (CTA).
- ✓ **Compatibilidad y accesibilidad:** El vídeo debe silenciarse automáticamente y no reproducirse en bucle en dispositivos móviles.
- ✓ **Carga optimizada:** Implementar una imagen de fondo para los casos en los que el vídeo no se cargue.
- ✓ **Adaptabilidad:** La sección debe ser 100% responsive y no afectar la usabilidad del sitio.